

Lab12-LinuxKernelModule2

目标

实现一个Linux模块

一个例子

编写内核hellokmodule.c

代码块

```
1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched/task.h>
5  #include <linux/sched/signal.h>
6  #include <linux/jiffies.h>
7
8  /* init function */
9  static int __init hellokmodule_init(void)
10 {
11     printk(KERN_ALERT "ALAL: simple module initialized\n");
12     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
13     return 0;
14 }
15
16 /* exit function - logs that the module is being removed */
17 static void __exit hellokmodule_exit(void)
18 {
19     printk(KERN_ALERT "ALAL: simple module is being unloaded\n");
20     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
21 }
22
23 module_init(hellokmodule_init);
24 module_exit(hellokmodule_exit);
25
26 MODULE_LICENSE ("GPL");
27 MODULE_AUTHOR ("LKD");
28 MODULE_DESCRIPTION ("Simple Kernel Module");
29 MODULE_VERSION("1.01");
```

创建Makefile文件

代码块

```
1  obj-m          := hellokmodule.o
2  KDIR           := /lib/modules/$(shell uname -r)/build
3  PWD            := $(shell pwd)
4
5  all:
6      $(MAKE) -C $(KDIR) M=$(PWD) modules
7
8  clean:
9      $(MAKE) -C $(KDIR) M=$(PWD) clean
10
11 testload:
12     sudo dmesg -C
13     sudo insmod hellokmodule.ko
14     dmesg | grep ALAL
15
16 testunload:
17     sudo dmesg -C
18     sudo rmmod hellokmodule
19     dmesg | grep ALAL
```

编译：

代码块

```
1  make
```

执行（插入模块）：

代码块

```
1  sudo make testload  # 应看到两行 ALAL 日志
```

执行（卸载模块）：

代码块

```
1  sudo make testunload # 应看到卸载日志
```

文件操作相关函数

与用户态 `fopen/fread/fwrite/fclose` 不同，内核态使用专属文件操作接口：

1. `filp_open()` - 内核态文件打开 / 创建函数

核心作用：在 Linux 内核态中打开一个已存在的文件，或创建一个新文件，返回内核专属的文件操作句柄（`struct file*`），替代用户态的 `fopen()`。

- **函数原型**（简化版，内核头文件 `<linux/fs.h>` 中定义）：

代码块

```
1 struct file *filp_open(const char *filename, int flags, umode_t mode);
```

- **参数说明：**

- `filename`：文件路径（绝对路径推荐，内核态无用户态当前工作目录概念）
- `flags`：文件打开标志，与用户态 `open()` 标志一致，常用值：
 - `O_RDONLY`：只读模式
 - `O_WRONLY`：只写模式
 - `O_RDWR`：读写模式
 - `O_CREAT`：文件不存在时创建文件（需配合 `mode` 参数使用）
 - `O_TRUNC`：文件存在时截断文件（清空原有内容）
 - `O_APPEND`：追加写入模式
- `mode`：文件权限掩码（仅当 `flags` 包含 `O_CREAT` 时有效），如 `0644`（用户可读可写，组可读，其他可读）、`0755`（用户可读可写可执行，组和其他可读可执行）

- **返回值：**

- 成功：返回有效的 `struct file*` 指针（内核态文件句柄）
- 失败：返回错误指针（需用 `IS_ERR()` 判断，`PTR_ERR()` 获取具体错误码）

2. `vfs_read()` - 内核态文件读取函数

核心作用：从 `filp_open()` 返回的内核文件指针中读取数据到内核缓冲区，替代用户态的 `fread()/read()`，是虚拟文件系统（VFS）层的通用读取接口，支持所有文件系统（ext4、tmpfs 等）。

- **函数原型**（简化版，内核头文件 `<linux/fs.h>` 中定义）：

代码块

```
1 ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos);
```

- **关键说明：**

- a. `file` : `filp_open()` 返回的有效 `struct file*` 指针（源文件句柄）
- b. `buf` : 数据缓冲区指针（**注意：**原型中默认是 `__user` 修饰，标识用户态缓冲区；若使用内核态缓冲区，需去掉该修饰或直接使用内核内存（如静态数组、`kmalloc()` 分配的内存）
- c. `count` : 期望读取的字节数
- d. `pos` : 文件读写偏移量指针（`loff_t` 类型），内核态无自动偏移机制，需手动维护和更新该值
- e. 返回值：成功返回实际读取的字节数；返回 0 表示读取到文件末尾；返回负数表示读取失败

3. `vfs_write()` - 内核态文件写入函数

核心作用：将内核缓冲区中的数据写入到 `filp_open()` 返回的内核文件指针中，替代用户态的 `fwrite()` / `write()`，同样是 VFS 层通用写入接口。

- **函数原型**（简化版，内核头文件 `<linux/fs.h>` 中定义）：

代码块

```
1  ssize_t vfs_write(struct file *file, const char __user *buf, size_t count,
    loff_t *pos);
```

- **关键说明：**

- a. `file` : `filp_open()` 返回的有效 `struct file*` 指针（目标文件句柄）
- b. `buf` : 待写入的数据缓冲区指针（同样支持 `__user` 用户态缓冲区和内核态缓冲区）
- c. `count` : 期望写入的字节数
- d. `pos` : 文件读写偏移量指针，需手动维护（写入后需根据实际写入字节数更新）
- e. 返回值：成功返回实际写入的字节数；返回负数表示写入失败（如磁盘满、权限不足）
- f. 注意：实际写入字节数可能小于期望写入字节数（非致命错误，可重试写入）

4. `filp_close()` - 内核态文件关闭函数

核心作用：关闭 `filp_open()` 打开的内核文件指针，释放对应的文件资源（文件句柄、内核缓存等），替代用户态的 `fclose()` / `close()`。

- **函数原型**（简化版，内核头文件 `<linux/fs.h>` 中定义）：

代码块

```
1  int filp_close(struct file *file, fl_owner_t id);
```

- **关键说明：**

- a. `file` : `filp_open()` 返回的 `struct file*` 指针（允许传入错误指针，内部会做判
- b. `id` : 文件所有者标识，内核态一般传入 `NULL` 或 `current->files`（当前进程文件描述符表），日常使用直接传 `NULL` 即可
- c. 返回值：成功返回 0；失败返回负数错误码（一般无需关注返回值，传入有效指针时极少失败）
- d. 必须调用：打开的文件若不调用 `filp_close()` 关闭，会导致内核资源泄漏，长期运行可能引发系统异常

实验流程

写一个C程序使用Linux内核机制复制文件

- 参考hellokmodule模块编写内核文件和Makefile文件；
- 准备源文件（加入你的学号姓名）；

代码块

```
1 echo "Hello Linux Kernel File Copy! This is a test file." >
  /home/user/source.txt
```

- 编译并执行内核模块；
- 验证复制结果；

代码块

```
1 cat /home/user/dest.txt
```

- 卸载内核模块

根据实验过程撰写实验报告